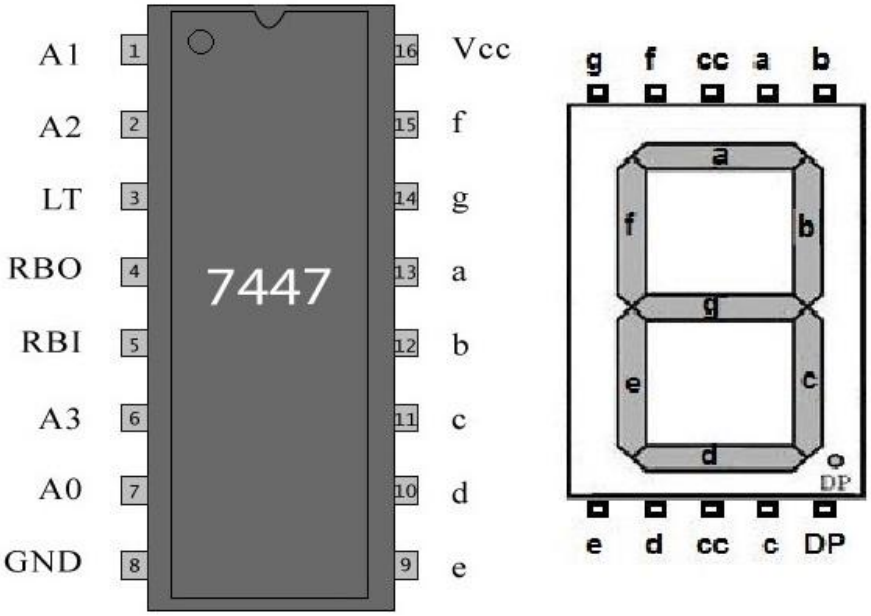



```
// 定義 LED 引腳陣列
const int ledPins[] = {4, 5, 6, 7, 8};
const int ledCount = 5;

void setup() {
 // 將所有 LED 引腳設定為輸出模式
 for (int i = 0; i < ledCount; i++) {
  pinMode(ledPins[i], OUTPUT);
 }
}

void loop() {
 // 依序點亮每一顆 LED
 for (int i = 0; i < ledCount; i++) {
  digitalWrite(ledPins[i], HIGH);  // 點亮當前 LED
  delay(200);                 // 延時 200 毫秒
  digitalWrite(ledPins[i], LOW);  // 熄滅當前 LED
 }
}
```

時間中斷

```
#include <TimerOne.h>

const int ledPins[] = {4, 5, 6, 7, 8};
const int ledCount = 5;

// 用 volatile 宣告在中斷服務程式中會改變的變數
volatile int currentLed = 0;

void setup() {
 for (int i = 0; i < ledCount; i++) {
   pinMode(ledPins[i], OUTPUT);
 }

 // 初始化 Timer1，設定週期為 200,000 微秒（等於 200 毫秒）
 Timer1.initialize(200000);

 // 設定中斷觸發時要執行的函式（ISR）
 Timer1.attachInterrupt(timerISR);

 // 先點亮第一顆燈
 digitalWrite(ledPins[currentLed], HIGH);
}

void loop() {
 // 主程式完全是空的，微處理器此時可以處理其他雜事
}

// 時間中斷服務程式 (ISR)
void timerISR() {
 // 1. 熄滅目前這顆燈
 digitalWrite(ledPins[currentLed], LOW);

 // 2. 切換到下一顆燈的索引 (0 -> 1 -> 2 -> 3 -> 4 -> 0)
 currentLed = (currentLed + 1) % ledCount;

 // 3. 點亮新的一顆燈
 digitalWrite(ledPins[currentLed], HIGH);
}
```

外部中斷

```
const int ledPins[] = {4, 5, 6, 7, 8};
const int ledCount = 5;
const int buttonPin = 2; // Uno 的外部中斷 0 專用引腳

volatile int currentLed = 0;
volatile bool direction = true; // true = 正向 (0->4), false = 反向 (4->0)

void setup() {
 for (int i = 0; i < ledCount; i++) {
  pinMode(ledPins[i], OUTPUT);
 }

 // 設定按鈕引腳為輸入上拉模式（平常為高電位，按下變低電位）
 pinMode(buttonPin, INPUT_PULLUP);

 // 設定外部中斷：當 Pin 2 偵測到下降沿（FALLING，即按鈕被按下）時，
觸發 buttonISR
 attachInterrupt(digitalPinToInterrupt(buttonPin), buttonISR, FALLING);
}

void loop() {
 // 點亮當前的 LED
 digitalWrite(ledPins[currentLed], HIGH);
 delay(200);
 digitalWrite(ledPins[currentLed], LOW);

 // 根據 direction 變數決定下一次換哪顆燈亮
 if (direction) {
  currentLed = (currentLed + 1) % ledCount; // 正向
 } else {
  currentLed = (currentLed - 1 + ledCount) % ledCount; // 反向
 }
}

// 外部中斷服務程式 (ISR)
void buttonISR() {
 // 只要按鈕一按下去，不管 loop 走到哪，立刻反轉方向
 direction = !direction;
}
```

「**集、基、射**」通常是指雙極性接面電晶體（BJT，俗稱三極體）的三個主要電極引腳： [1]

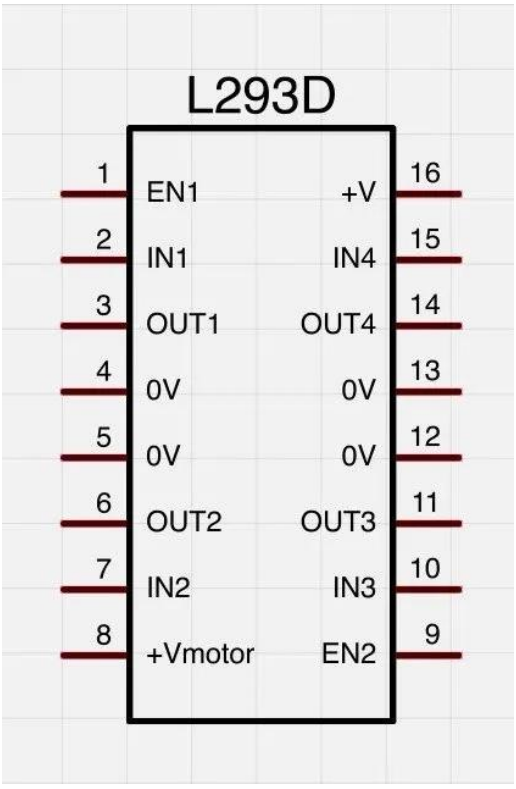
- **集極 (Collector, C)**：主要的電流輸出端。輸出
- **基極 (Base, B)**：控制訊號的輸入端，相當於水龍頭的開關。 輸入
- **射極 (Emitter, E)**：載子（電子或電洞）的發射或流出端。接地

```
void setup() {
  pinMode(3, OUTPUT);
}

void loop() {

  analogWrite(3, analogRead(A0) / 4);

}
```



```
// 定義引腳
const int potPin = A0;     // 電位器接 A0
const int buttonPin = 4;   // 按鈕接 D4
const int enablePin = 9;   // L293NE Pin 1 (PWM 調速)
const int in1Pin = 2;      // L293NE Pin 2
const int in2Pin = 3;      // L293NE Pin 7

// 變數宣告
int potValue = 0;         // 儲存電位器讀取值 (0-1023)
int motorSpeed = 0;       // 轉換後的馬達速度 (0-255)
bool isForward = true;    // 馬達方向狀態，true 為正轉，false 為反轉
bool lastButtonState = HIGH; // 記錄上一次按鈕的狀態（預設上拉為 HIGH）

void setup() {
 // 設定引腳模式
 pinMode(buttonPin, INPUT_PULLUP); // 使用內建上拉電阻，沒按下時為
HIGH，按下時為 LOW
 pinMode(enablePin, OUTPUT);
 pinMode(in1Pin, OUTPUT);
 pinMode(in2Pin, OUTPUT);
}

void loop() {
 // --- 1. 轉速控制 ---
 potValue = analogRead(potPin);              // 讀取電位器 (0 ~ 1023)
 motorSpeed = map(potValue, 0, 1023, 0, 255);   // 將範圍轉換為 PWM 範圍 (0
~ 255)
 analogWrite(enablePin, motorSpeed);         // 輸出 PWM 訊號控制轉速

 // --- 2. 按鈕狀態偵測（正反轉切換） ---
 bool currentButtonState = digitalRead(buttonPin);

 // 偵測按鈕被「按下」的那一瞬間（從 HIGH 變成 LOW）
 if (lastButtonState == HIGH && currentButtonState == LOW) {
  isForward = !isForward; // 反轉方向狀態
  delay(50);          // 簡單的按鈕彈跳（Debounce）延遲
 }
 lastButtonState = currentButtonState; // 更新按鈕狀態

 // --- 3. 驅動馬達轉向 ---
 if (isForward) {
```

```
  // 正轉
  digitalWrite(in1Pin, HIGH);
  digitalWrite(in2Pin, LOW);
 } else {
  // 反轉
  digitalWrite(in1Pin, LOW);
  digitalWrite(in2Pin, HIGH);
 }
}
```

